FUNDAMENTALS OF MACHINE LEARNING LAB RECORD

Name: KESAVARAPU AKHIL
Register Number: 11259A213
Department: Computer Science and Engineering
Semester: II Year / III Semester

Experiment Index

1. Experiment 1: Data Preprocessing in a Given Dataset
2. Experiment 2: Data Splitting and Cross-Validation
3. Experiment 3: Implement Linear Regression
4. Experiment 4A: Classification Using K-Nearest Neighbors (KNN)
5. Experiment 4B: Classification Using Support Vector Machine (SVM)
6. Experiment 5: Implement K-Means Clustering
7. Experiment 6: Implement Hierarchical Clustering
8. Experiment 7: Random Forest Classification and Performance Analysis
9. Experiment 8: AdaBoost Classification and Performance Analysis
10. Experiment 9: Basic Reinforcement Learning Using Q-Learning
11. Experiment 10: Neural Network for Binary Classification

Experiment 1: Data Preprocessing in a Given Dataset

Aim

To inspect a dataset, handle missing values, and standardize numerical features using Python.

Algorithm

12. Import pandas and StandardScaler.
13. Read data.csv into a DataFrame.
14. Display the first five rows and count missing values.
15. Fill missing numeric values with their column means.
16. Select the numerical feature columns and standardize them.
17. Display the normalized data.

Program

Program Screenshot — Experiment 1

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("data.csv")
print("Original Dataset:")
print(data.head())
print("\\nMissing Values:")
print(data.isnull().sum())

data.fillna(data.mean(numeric_only=True), inplace=True)
X = data[["Hours_Studied", "Attendance", "Assignments", "Marks"]]
scaler = StandardScaler()
X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)

print("\\nNormalized Dataset:")
print(X_scaled.head())
```

Output

Output Screenshot — Experiment 1

```
Original Dataset:
   Hours_Studied  Attendance  Assignments  Marks
0              2          70            6     35
1              4          75            7     50
2              6          80            8     65
3              8          90            9     80
4             10          95           10     95

Missing Values:
Hours_Studied    0
Attendance       0
Assignments      0
Marks            0

Normalized Dataset:
   Hours_Studied  Attendance  Assignments  Marks
0          -1.41       -1.45        -1.41  -1.41
1          -0.71       -0.87        -0.71  -0.71
... (values depend on data.csv)
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended

machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 2: Data Splitting and Cross-Validation

Aim

To split a dataset into training and testing sets and evaluate a K-Nearest Neighbors classifier using 5-fold cross-validation.

Algorithm

18. Load data.csv and separate input features from the target label.
19. Split the data into 80% training and 20% testing sets using a fixed random state.
20. Print the training and testing sample counts.
21. Create a KNN classifier with k = 3.
22. Run 5-fold cross-validation on the dataset.
23. Display the fold scores and their average.

Program

Program Screenshot — Experiment 2

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.neighbors import KNeighborsClassifier

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print("Training Data Size:", len(X_train))
print("Testing Data Size:", len(X_test))

model = KNeighborsClassifier(n_neighbors=3)
scores = cross_val_score(model, X, y, cv=5)
print("\\nCross Validation Scores:")
print(scores)
print("Average Accuracy:", scores.mean())
```

Output

Output Screenshot — Experiment 2

```
Training Data Size: 8
Testing Data Size: 2

Cross Validation Scores:
[0.50 1.00 1.00 0.50 1.00]
Average Accuracy: 0.80

Illustrative output only; scores depend on data.csv.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 3: Implement Linear Regression

Aim

To train a Linear Regression model to predict marks from study hours and evaluate its performance.

Algorithm

24. Read the dataset and select Hours_Studied as the input feature and Marks as the target.
25. Split the data into training and testing sets.
26. Create and train a Linear Regression model.
27. Predict marks for the test set.
28. Calculate Mean Squared Error and R^2 score.
29. Plot actual values and the regression line.

Program

Program Screenshot — Experiment 3

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

data = pd.read_csv("data.csv")
X = data[["Hours_Studied"]]
y = data["Marks"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))
print("Predicted Marks:", y_pred)

plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Regression Line")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.legend()
plt.show()
```

Output

Output Screenshot — Experiment 3

```
Mean Squared Error: (computed from test data)
R2 Score: (computed from test data)
Predicted Marks: [predicted values]

A scatter plot and fitted regression line are displayed.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 4A: Classification Using K-Nearest Neighbors (KNN)

Aim

To build a KNN classification model and evaluate it using accuracy, confusion matrix, and classification report.

Algorithm

30. Load the dataset and select Hours_Studied, Attendance, and Assignments as features.
31. Select Result as the target variable.
32. Split the dataset into training and testing sets.
33. Train a KNN classifier with three neighbors.
34. Predict the test labels.
35. Print accuracy, confusion matrix, and classification report.

Program

Program Screenshot — Experiment 4

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output

Output Screenshot — Experiment 4

```
Accuracy: (computed from test data)

Confusion Matrix:
[[... ...]
 [... ...]]

Classification Report:
              precision  recall  f1-score
Fail              ...      ...      ...
Pass              ...      ...      ...

Exact metrics depend on data.csv.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution

environment.

Experiment 4B: Classification Using Support Vector Machine (SVM)

Aim

To implement an SVM classifier and evaluate its classification performance.

Algorithm

36. Read data.csv and choose the input features and target label.
37. Split the dataset into training and testing sets.
38. Create an SVM classifier using a linear kernel.
39. Train the model and predict test labels.
40. Calculate accuracy, confusion matrix, and classification report.

Program

Program Screenshot — Experiment 5

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = SVC(kernel="linear")
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output

Output Screenshot — Experiment 5

```
Accuracy: (computed from test data)

Confusion Matrix:
[[... ...]
 [... ...]]

Classification Report:
Precision, recall, and F1-score are displayed for each class.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 5: Implement K-Means Clustering

Aim

To group data points into three clusters using the K-Means clustering algorithm and visualize the clusters and centroids.

Algorithm

41. Load the dataset and select Hours_Studied and Marks.
42. Create a K-Means model with three clusters.
43. Fit the model to the selected features.
44. Add the cluster labels to the dataset and print it.
45. Plot the data points by cluster and mark the centroids.

Program

Program Screenshot — Experiment 6

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Marks"]]
model = KMeans(n_clusters=3, random_state=42, n_init=10)
model.fit(X)
clusters = model.labels_
data["Cluster"] = clusters
print(data)

plt.scatter(X["Hours_Studied"], X["Marks"], c=clusters)
plt.scatter(model.cluster_centers_[:, 0],
            model.cluster_centers_[:, 1],
            marker="X", s=200, label="Centroids")
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("K-Means Clustering")
plt.legend()
plt.show()
```

Output

Output Screenshot — Experiment 6

```
   Hours_Studied  Marks  Cluster
0              2     35        1
1              4     50        1
2              6     65        0
3              8     80        2
4             10     95        2

A cluster scatter plot with centroid markers is displayed.
Cluster IDs can vary between runs/datasets.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 6: Implement Hierarchical Clustering

Aim

To perform agglomerative hierarchical clustering and visualize the dendrogram and resulting clusters.

Algorithm

46. Load the dataset and select Hours_Studied and Marks.
47. Compute a linkage matrix using Ward's method.
48. Draw a dendrogram to visualize hierarchical grouping.
49. Apply AgglomerativeClustering with three clusters.
50. Add cluster labels to the dataset and print the results.
51. Plot the clustered points.

Program

Program Screenshot — Experiment 7

```
import pandas as pd
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Marks"]]
linked = linkage(X, method="ward")

plt.figure(figsize=(8, 5))
dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Data Points")
plt.ylabel("Euclidean Distance")
plt.show()

model = AgglomerativeClustering(n_clusters=3)
clusters = model.fit_predict(X)
data["Cluster"] = clusters
print(data)

plt.scatter(X["Hours_Studied"], X["Marks"], c=clusters)
plt.xlabel("Hours Studied")
plt.ylabel("Marks")
plt.title("Hierarchical Clustering")
plt.show()
```

Output

Output Screenshot — Experiment 7

```
The program displays:
1. A dendrogram showing hierarchical merges.
2. A table containing the original features and Cluster labels.
3. A scatter plot of the three clusters.

Cluster labels depend on the dataset.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 7: Random Forest Classification and Performance Analysis

Aim

To train a Random Forest classifier, evaluate its performance, and visualize feature importance.

Algorithm

52. Load data.csv and select the three input features and Result target.
53. Split the data into training and testing sets.
54. Create a Random Forest classifier with 100 trees.
55. Train the classifier and predict the test labels.
56. Print accuracy, confusion matrix, and classification report.
57. Plot feature importance values.

Program

Program Screenshot — Experiment 8

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.bar(X.columns, model.feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("Random Forest Feature Importance")
plt.show()
```

Output

Output Screenshot — Experiment 8

```
Accuracy: (computed from test data)

Confusion Matrix:
[[... ...]
 [... ...]]

Classification report and a feature-importance bar chart are displayed.
Metrics depend on data.csv.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 8: AdaBoost Classification and Performance Analysis

Aim

To implement the AdaBoost ensemble learning technique and evaluate the classifier.

Algorithm

58. Load the dataset and select input features and target labels.
59. Split the dataset into training and testing subsets.
60. Create an AdaBoost classifier with 50 estimators.
61. Train the model and predict test labels.
62. Evaluate using accuracy, confusion matrix, and classification report.
63. Plot the feature importance values.

Program

Program Screenshot — Experiment 9

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = AdaBoostClassifier(n_estimators=50, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("\\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
print("\\nClassification Report:")
print(classification_report(y_test, y_pred))

plt.bar(X.columns, model.feature_importances_)
plt.xlabel("Features")
plt.ylabel("Importance")
plt.title("AdaBoost Feature Importance")
plt.show()
```

Output

Output Screenshot — Experiment 9

```
Accuracy: (computed from test data)

Confusion matrix and classification report are printed.
A feature-importance bar chart is displayed.
Exact values depend on data.csv.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 9: Basic Reinforcement Learning Using Q-Learning

Aim

To implement Q-learning using a reward matrix and display the learned Q-table.

Algorithm

64. Define the reward matrix for the six states.
65. Initialize the Q-table with zeros.
66. Set the discount factor gamma to 0.8 and episodes to 1000.
67. For each episode, choose a random starting state.
68. Choose an allowed action, update the Q-value using the reward and future maximum Q-value, and move to the next state.
69. Stop an episode when the goal state is reached and print the Q-table.

Program

Program Screenshot — Experiment 10

```
import numpy as np

R = np.array([
    [-1, -1, -1, -1, 0, -1],
    [-1, -1, -1, 0, -1, 100],
    [-1, -1, -1, 0, -1, -1],
    [-1, 0, 0, -1, 0, -1],
    [0, -1, -1, 0, -1, 100],
    [-1, 0, -1, -1, 0, 100]
])

Q = np.zeros_like(R, dtype=float)
gamma = 0.8
episodes = 1000

for _ in range(episodes):
    state = np.random.randint(0, 6)
    steps = 0
    while state != 5 and steps < 100:
        actions = np.where(R[state] >= 0)[0]
        action = np.random.choice(actions)
        next_state = action
        Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])
        state = next_state
        steps += 1

print("Q-Table:\\n")
print(Q)
```

Output

Output Screenshot — Experiment 10

```
Q-Table:

[[ ... learned Q-values ... ]
 [ ... learned Q-values ... ]
 [ ... learned Q-values ... ]
 [ ... learned Q-values ... ]
 [ ... learned Q-values ... ]
 [ ... learned Q-values ... ]

Q-values are learned over training episodes and can vary
because the starting states and actions are random.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.

Experiment 10: Neural Network for Binary Classification

Aim

To build and train a simple neural network to classify records as Fail or Pass and evaluate its accuracy.

Algorithm

70. Load the dataset and select Hours_Studied, Attendance, and Assignments as features.
71. Convert Fail and Pass labels to numeric values 0 and 1.
72. Split the dataset into training and testing sets.
73. Standardize features using StandardScaler.
74. Build a neural network with two ReLU hidden layers and a sigmoid output layer.
75. Compile and train the model for 50 epochs.
76. Predict test labels, calculate accuracy, and print the result.

Program

Program Screenshot — Experiment 11

```
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score

data = pd.read_csv("data.csv")
X = data[["Hours_Studied", "Attendance", "Assignments"]]
y = data["Result"].map({"Fail": 0, "Pass": 1})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(8, activation="relu", input_shape=(3,)),
    tf.keras.layers.Dense(4, activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])
model.compile(optimizer="adam", loss="binary_crossentropy",
              metrics=["accuracy"])
model.fit(X_train, y_train, epochs=50, validation_split=0.2, verbose=1)

y_pred = (model.predict(X_test) > 0.5).astype(int).ravel()
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output

Output Screenshot — Experiment 11

```
Epoch 50/50
... training/validation metrics ...

Accuracy: (computed from test data)

Training progress and the final accuracy are displayed.
Results vary with the dataset and training run.
```

Result

Thus, the experiment was carried out according to the algorithm. The program demonstrates the intended machine-learning/data-processing technique; exact numerical results depend on the dataset and execution environment.